

2026 腾讯 mini 项目课程前置学习笔记

姓名	庄锐敏	学号	202400203093	班级	数科三班
报名项目	性能工程 Skill 评估专项				
选择此项目的原因	选择本项目主要是因为它与我已有的学习和实践经历比较契合，同时也能进一步提升我在 Skill、Agent 评估和自动化测试方面的能力。 此前我做过 AFC 自动事实核查项目，对问题拆解、证据检索、事实判断、0/1/2 评分、主需/次需错误区分等流程有一定基础。本项目中的 tracingclaw_finance Skill 与金融事实核查高度相关，可以让我把之前 AFC 项目中的事实核查经验迁移到金融场景中，进一步学习金融数据源选择、口径一致性和事实校验方法。 我选择这个项目，是希望不仅学习如何使用 Skill，更希望从评估者的角度理解一个 Skill 是否稳定、可靠、可复用。通过设计 ideal_state、rubrics、testcases 和 meta-testcase，我可以系统提升自己在 Agent/Skill 评估、自动化测试、事实核查、性能分析和数据驱动报告生成方面的综合能力。				
项目分析	第一在于要从“使用 Skill”转变为“评估 Skill”。这个项目不是简单地让 Skill 完成任务，而是要设计 ideal_state、rubrics、testcases 和 meta-testcase，判断一个 Skill 是否稳定、可靠、可复用。因此需要先理解 Skill 的工作流、输入输出、数据源边界和失败处理机制，再把这些要求转化成可执行、可判分的测试用例。 第二在于测试用例不能只覆盖正常场景，还要覆盖失败路径、边界条件和红线违规。例如 db_report 不能只测试正常生成报告，还要测试缺少数据源、字段缺失、内部接口调用、编造性能指标等异常情况；tracingclaw_finance 也不能只测试普通财报核查，还要覆盖市场混淆、币种混淆、财年口径错误、工具查不到数据时是否编造等问题。 第三在于 Rubrics 的设计要足够具体。评分标准不能写成“结果合理”“内容完整”这种模糊描述，而是要明确什么情况扣分、什么情况属于严重错误、什么情况可以直接判失败。例如编造 TPS/QPS/P95、混用金融数据口径、跳过规定数据源等都应该作为红线问题处理。 第四在于两个 Skill 的知识背景不同。db_report 涉及数据库性能测试、TPS/QPS、P95/P99、报告类型和数据质量门控；tracingclaw_finance 涉及金融事实核查、westock-data 和 mx-finance-search 工具路由、财报/行情/公告/研报等不同数据源，以及 0/1/2 评分逻辑。因此需要分别理解两个 Skill 的业务链路，再设计对应的评估方案。 第五在于 meta-testcase 的设计。它不是测试 Skill 本身，而是反过来检查自己设计的测试集是否全面、是否可执行、是否能稳定判分。这要求我不仅要会写测试用例，还要能从评估体系的角度检查覆盖率、判分清晰度和红线完整性。				
岗位职责分析	我可以承担“撰写测试用例”相关工作，并可以协助完成“撰写测试用例的测试用例”。 在撰写测试用例方面，我可以负责阅读 Skill 文档和参考资料，梳理 Skill 的输入、输出、执行流程、能力边界和失败场景，并将这些内容转化为可执行、可判定的 testcase。例如针对 db_report，我可以设计缺少数据源、文件不存在、关键字段缺失、本地 log 单次报告、records JSON 对比报告、iteration 历史趋势、custom 专项分析、三格式交付一致性等测试用例；针对 tracingclaw_finance，我可以结合金融事实核查流程，设计财报数字核查、行情数据核查、公告/研报交叉验证、市场/币种/时间口径混淆、数据不足禁止编造等测试用例。 同时，我也可以协助“撰写测试用例的测试用例”，也就是 meta-testcase 的设计。该部分主要用于检查我们写出的测试用例本身是否字段完整、覆盖充分、判定规则明确、是否覆盖正常路径、失败路径、边界条件和红线行为。通过 meta-testcase，可以反向保证测试集质量，而不是只关注 Skill 是否能通过单				

	个测试。
搜集到的学习资料	OpenAI 官方: Skills in ChatGPT: https://help.openai.com/zh-hans-cn/articles/20001066-skills-in-chatgpt?utm_source=chatgpt.com OpenAI Codex Skills 官方文档: https://developers.openai.com/codex/skills?utm_source=chatgpt.com 如何快速学习一个新 Skill: https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents 测试用例的质量 https://promptfoo.org.cn/docs/configuration/expected-outputs/model-graded/llm-rubric/
制定的学习计划	学习计划: 性能工程 Skill 评估专项 第 1 天: 理解项目整体要求 学习本项目的任务目标, 明确本项目不是直接开发 Skill, 而是围绕 db_report 和 tracingclaw_finance 两个 Skill 设计评估方案。重点理解 ideal_state、logic_chain、rubrics、testcases、meta-testcase 的作用, 以及测试用例需要覆盖正常路径、失败路径、边界条件和红线行为。 第 2 天: 学习 db_report Skill 的业务链路 阅读 db_report 相关材料, 理解数据库性能报告生成流程, 重点学习 single、comparison、iteration、custom 等报告类型, 以及 TPS、QPS、P95、P99、响应时间、吞吐量、错误率等性能指标。重点关注缺少数据、字段不完整、数据源不可读、内部接口越权调用和编造性能指标等失败场景。 第 3 天: 完成 db_report 的评估方案设计 围绕 db_report 设计理想状态、评分标准和测试用例, 覆盖正常报告生成、对比报告、迭代报告、缺数据拒绝、字段缺失、最小交付物检查、数据质量门控和红线行为。通过这一阶段学习如何把 Skill 的执行流程转化为可判定的 testcase。 第 4 天: 学习 tracingclaw_finance Skill 的金融事实核查流程 结合此前 AFC 自动事实核查项目经验, 学习 tracingclaw_finance 的核查链路, 包括输入识别、候选回答识别、核验点拆解、主需/次需区分、证据获取、0/1/2 真实性评分和修正参考答案生成。重点理解金融事实核查与普通事实核查的区别。 第 5 天: 学习金融数据源路由与口径校验 学习 westock-data 与 mx-finance-search 的分工: 财报、行情、公司信息等结构化金融事实优先使用 westock-data; 公告、新闻、研报、政策和事件背景优先使用 mx-finance-search。同时重点学习市场、时间、币种、单位、财年/自然年、单季/累计、同比/环比、实时/延时等金融口径问题。 第 6 天: 完成 tracingclaw_finance 的评估方案设计 完成 tracingclaw_finance 的 ideal_state、logic_chain、rubrics.yaml、testcases.yaml、meta_testcases.md 和 README.md。测试用例覆盖财报数字核查、行情涨跌幅核查、公告与研报观点交叉验证、市场/时间/币种混用、数据不足禁止编造、EM_API_KEY 缺失、westock-data 不可用、评分边界和投资建议边界等场景。 第 7 天: 统一整理提交材料并进行自测准备 对两个 Skill 的提交文件进行统一整理, 检查 YAML 语法、字段完整性、Rubrics 权重、coverage 覆盖、meta-testcase 自检规则和 README 说明。根据学长建议, 准备用 Codex 作为自测执行 Agent, 生成

		HTML 格式测试报告，用于验证 <code>testcase</code> 是否可执行、可判定，并区分静态质量检查、过程逻辑检查和依赖环境未满足等情况。	
		目前已经完成两个 <code>Skill</code> 的主要评估方案设计，掌握了从 <code>Skill</code> 说明到理想态、评分标准、测试用例、元测试和提交说明的完整设计流程。基于 <code>Codex</code> 进行自测执行，生成 HTML 测试报告。	
学习心得	1	在做题过程中遇到的问题	在设计 <code>db_report</code> 的 <code>testcase</code> 时，我一开始倾向于把每一个小能力都单独写成一条用例，例如 AND 条件解析、OR 条件解析、三格式交付、L1/L2/L3 洞察等都分别拆开。但是这样会导致用例数量变多，同时前几条用例主要集中在输入解析和数据接入阶段，对性能分析、图表生成、报告交付这些后续阶段覆盖不够。
		自己对问题的理解或预判	我的理解是，测试用例不是越多越好，而是要看每条用例能不能覆盖更多关键链路。项目要求我们设计测试用例并验证 <code>Skill</code> 的整体交付能力；如果用例过碎，就会出现“数量多但覆盖低”的问题；应该优先设计复合型用例，让一条正常路径用例尽量覆盖多种场景。
		曾采取的行动或措施	我重新调整了 <code>testcase</code> 的设计思路，把用例分成正常端到端用例、失败路径用例、红线用例和专项检查用例。正常路径用例尽量服务最终测试报告输出，失败路径用例则重点验证是否正确停止、拒绝或提示用户补充数据。
		AI 对此问题的回答	GPT 建议我不要为了凑数量写碎片化用例，而是用更少的高价值复合用例覆盖更多阶段。例如 DB-01 本地 <code>log</code> 单次报告可以同时覆盖输入解析、 <code>log</code> 解析、数据质量门控、五场景分析、图表生成和三格式交付；DB-04 <code>custom</code> 专项分析可以同时覆盖自定义问题、AND/OR 条件、P95 异常分析和 L1/L2/L3 洞察。
		针对 AI 的回答，是否向导师进一步提问？	AI 的回答已经解决了我的困惑，后续只需要在 <code>testcase</code> 中体现“少用例、多覆盖”的设计思想即可。
	2	在做题过程中遇到的问题	在设计 <code>rubrics</code> 时，我不确定是否应该继续细分评分维度。例如是否要把图表质量、三格式排版、L1/L2/L3 洞察、最小交付检查都拆成单独维度。
		自己对问题的理解或预判	如果 <code>rubrics</code> 拆得过细，可能会看起来更完整，但也会导致重复扣分和自动化判定困难。 <code>Rubrics</code> 需要覆盖 <code>Skill</code> 的完整质量要求；如果每个细节都拆成一级维度，会导致评分体系复杂且不稳定；一级维度应保持稳定，细节放到 <code>testcase</code> 的 <code>judge</code> 中检查。
		曾采取的行动或措施	我保留了 R1 到 R6 六个主要维度，分别对应输入解析、数据接入、数据质量门控、性能分析与洞察、图表与三格式交付、红线与拒绝机制。同时检查每个维度下 <code>partial_deductions</code> 的扣分合计是否等于该维度总分。
		AI 对此问题的回答	I 建议不要再继续拆分一级维度。当前 R1~R6 已经能够对应 <code>db_report</code> 的完整执行链路，更细的要求，例如 <code>single</code> 至少 11 张图、 <code>comparison</code> 至少 17+N 张

			图、docx 字体、html 内联 CSS、无占位符等，更适合放进具体 testcase 的 judge 中，而不是继续拆 rubric 维度。
		针对 AI 的回答，是否向导师进一步提问？	暂时不需要。这个问题已经形成了比较清楚的处理原则：rubrics 保持粗粒度，testcases 负责细粒度检查。
3		在做题过程中遇到的问题	在 tracinglaw_finance 的测试设计中，如果任务需要使用 mx-finance-search 查询公告、新闻或研报，但当前环境缺少 EM_API_KEY，这种情况应该判定为 Skill 失败，还是判定为环境前提未满足？
		自己对问题的理解或预判	我一开始容易把“无法完成查询”直接理解为 Skill 没做好。但进一步分析后发现，mx-finance-search 本身依赖 EM_API_KEY，如果环境变量没有配置，Skill 无法完成资讯检索是合理的环境限制。真正需要测试的是：Skill 能不能识别当前环境前提不足，并正确停止，而不是假装已经查询成功。
		曾采取的行动或措施	我在 testcase 中专门设计了 EM_API_KEY 缺失场景，并把它作为环境失败路径进行测试。预期行为是：Skill 应该说明当前缺少必要环境配置，不能完成公告、新闻或研报检索；不能编造资讯结果，也不能输出正式核查结论或正式评分。
		AI 对此问题的回答	缺少 EM_API_KEY 时，不能直接判定为 Skill 事实核查逻辑失败，而应优先判定为环境前提未满足。但如果 Skill 在缺少 key 的情况下仍声称完成了 mx-finance-search 查询，或者编造公告、新闻、研报结果，则应判定为严重失败
		针对 AI 的回答，是否向导师进一步提问？	老师回答说可根据 skill 提示配置 key